Requirements

- Install Python
- Install Django
- How to Create Django Project, Apps and do settings
- How to Create Templates and do settings

Django Template Language

Django’s template language is designed to strike a balance between power and ease. It’s designed to feel comfortable to those used to working with HTML.

***courseone*.html**

```
<html>
   <body>
              <h2> Course Name:{{nm}} Duration:{{du}} and Total Seats: {{st}}</h2>
     </body>
</html>
```

Jinja2 - Jinja is a modern and designer-friendly templating language for Python, modelled after Django’s templates. It is fast, widely used and secure with the optional sandboxed template execution environment.

python pip install jinja2

'BACKEND': 'django.template.backends.jinja2.Jinja2',

Variables

Variables look like this: **{{ variable }}** When the template engine encounters a variable, it evaluates that variable and replaces it with the result.

Rules:-

Variable names consist of any combination of alphanumeric characters and the underscore.

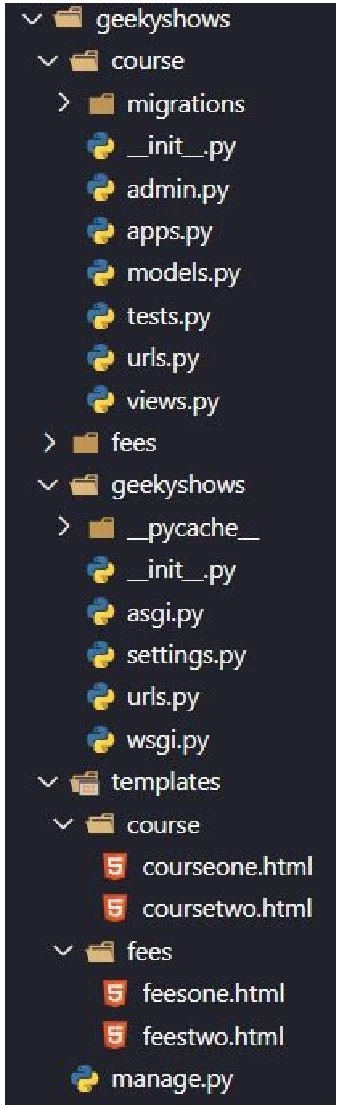
Variable name should not start with underscore.

Variable name can not have spaces or punctuation characters.

Syntax:- {{variable}}

Example:- {{nm}}, {{name1}}, {{first_name}}

Dynamic Template Files



views.py

```
from django.shortcuts import render
def learn_django(request):
        cname = ‘Django’
        duration = ‘4 Months’
        seats = 10
        django_details = {‘nm’:cname, ‘du’:duration, ‘st’:seats}
        return render(request, ‘course/courseone.html’, django_details)
```

templates/course

***courseone*.html**

```
<html>
  <body>
        <h2> Course Name:{{nm}} Duration:{{du}} and Total Seats: {{st}}</h2>
   </body>
</html>
```

Filters

When we need to modify variable before displaying we can use filters. Pipe '|' is used to apply filter.

Syntax:- {{variable | filter}}

Example:- {{name|upper}}

Some of filters take arguments.

Syntax:- {{variable | filter : argument}}

Example:- {{article|truncateword:40}}

Filter can be chained.

Syntax:- {{variable | filter | filter}}

Example:- {{article|upper}}

Example:- {{article|upper|truncateword:40}}

Filters

capfirst – It capitalizes the first character of the value. If the first character is not a letter, this filter has no effect.

Example:- {{value|capfirst}}

default - If value evaluates to False, uses the given default. Otherwise, uses the value.

Example:- {{ value|default:"nothing" }}

If value is "" (the empty string), the output will be nothing.

length - It returns the length of the value. This works for both strings and lists. The filter returns 0 for an undefined variable.

Example:- {{ value|length }}

lower - It converts a string into all lowercase.

Example:- {{ value|lower }}

Filters

upper - It converts a string into all uppercase.

Example:- {{ value|upper }}

slice - It returns a slice of the list. Uses the same syntax as Python’s list slicing.

Example:- {{ some_list|slice:":2" }}

truncatechars - It truncates a string if it is longer than the specified number of characters. Truncated strings will end with a translatable ellipsis character (“…”).

Argument: Number of characters to truncate to

Example:- {{ value|truncatechars:7 }}

truncatewords - It truncates a string after a certain number of words. Newlines within the string will be removed.

Argument: Number of words to truncate after

Example:- {{ value|truncatewords:2 }}

Filters

date – It formats a date according to the given format.

Example:- {{value|date:“D d M Y”}}

time – It formats a time according to the given format.

Example:- {{value|time:“H:i”}}

Day

Format Character	Description	Example
d	Day of the month, 2 digits with leading zeros	01 to 31
j	Day of the month without leading zeros	1 to 31
D	Day of the week, textual, 3 letters	Fri
l	Day of the week, textual, long	Friday
S	English ordinal suffix for day of the month, 2 characters	st, nd, rd or th
w	Day of the week, digits without leading zeros	0 (Sunday) to 6 (Saturday)
z	Day of the year	1 to 366

Week

Format Character	Description	Example
W	ISO-8601 week number of year, with weeks starting on Monday	1, 53

Month

Format Character	Description	Example
m	Month, 2 digits with leading zeros	01 to 12
n	Month without leading zeros	1 to 12
M	Month, textual, 3 letters	Jan
b	Month, textual, 3 letters, lowercase	jan
E	Month, locale specific alternative representation usually used for long date representation	'listopada' (for Polish locale, as opposed to 'Listopad')
F	Month, textual, long	January
N	Month abbreviation in Associated Press style. Proprietary extension	Jan., Feb., March, May
t	Number of days in the given month	28 to 31

Year

Format Character	Description	Example
y	Year, 2 digits	99
Y	Year, 4 digits	1999
L	Boolean for whether it’s a leap year	True or False
o	ISO-8601 week-numbering year, corresponding to the ISO-8601 week number (W) which uses leap weeks. See Y for the more common year format	1999

Time

Format Character	Description	Example
g	Hour, 12-hour format without leading zeros	1 to 12
G	Hour, 24-hour format without leading zeros	0 to 23
h	Hour, 12-hour format	01 to 12
H	Hour, 24-hour format	00 to 23
i	Minutes	00 to 59
s	Seconds, 2 digits with leading zeros	00 to 59
u	Microseconds	000000 to 999999
a	'a.m.' or 'p.m.' (Note that this is slightly different than PHP’s output, because this includes periods to match Associated Press style.)	a.m.
A	'AM' or 'PM'	AM
f	Time, in 12-hour hours and minutes, with minutes left off if they’re zero. Proprietary extension	1, 1:30
p	Time, in 12-hour hours, minutes and ‘a.m.’/’p.m.’, with minutes left off if they’re zero and the special-case strings ‘midnight’ and ‘noon’ if appropriate. Proprietary extension	1 a.m., 1:30 p.m., midnight, noon, 12:30 p.m.

Timezone

Format Character	Description	Example
e	Timezone name. Could be in any format, or might return an empty string, depending on the datetime	'', 'GMT', '-500', 'US/Eastern', etc
I	Daylight Savings Time, whether it’s in effect or not	1 or 0
O	Difference to Greenwich time in hours	+0200
T	Time zone of this machine	EST, MDT
Z	Time zone offset in seconds. The offset for timezones west of UTC is always negative, and for those east of UTC is always positive	-43200 to 43200

Date/Time

Format Character	Description	Example
c	ISO 8601 format. (Note: unlike others formatters, such as “Z”, “O” or “r”, the “c” formatter will not add timezone offset if value is a naive datetime	2008-01-02T10:30:00.000123+02:00, or 2008-01-02T10:30:00.000123 if the datetime is naive
r	RFC 5322 formatted date	'Thu, 21 Dec 2000 16:01:07 +0200'
U	Seconds since the Unix Epoch (January 1 1970 00:00:00 UTC)	

Predefined Formats

Format	Description	Example
DATE_FORMAT	Default: 'N j, Y'	Feb. 9, 2020
DATETIME_FORMAT	Default: 'N j, Y, P'	Feb. 9, 2020, 10 p.m.
SHORT_DATE_FORMAT	Default: 'm/d/Y'	12/31/2020
SHORT_DATETIME_FORMAT	Default: 'm/d/Y P'	12/31/2020 4 p.m.
TIME_FORMAT	Default: "H:i"	“01:24”

Example:- {{ value|date:"SHORT_DATE_FORMAT" }}

Example:- {{ value|time:“TIME_FORMAT" }}

Filters

floatformat

When used without an argument, rounds a floating-point number to one decimal place but only if there’s a decimal part to be displayed.

Value	Template	Output
56.24321	{{value|floatformat}}	56.2
56.00000	{{value|floatformat}}	56
56.37000	{{value|floatformat}}	56.4

If used with a numeric integer argument, floatformat rounds a number to that many decimal places.

Value	Template	Output
56.24321	{{value|floatformat:3}}	56.243
56.00000	{{value|floatformat:3}}	56.000
56.37000	{{value|floatformat:3}}	56.370

Filters

floatformat

Particularly useful is passing 0 (zero) as the argument which will round the float to the nearest integer.

Value	Template	Output
56.24321	{{value|floatformat:“0”}}	56
56.00000	{{value|floatformat:“0”}}	56
56.37000	{{value|floatformat:“0”}}	56

If the argument passed to floatformat is negative, it will round a number to that many decimal places but only if there’s a decimal part to be displayed.

Value	Template	Output
56.24321	{{value|floatformat:“–3 ”}}	56.243
56.00000	{{value|floatformat:“–3 ”}}	56
56.37000	{{value|floatformat:“–3 ”}}	56.370

if Tag

{% if %} tag - The {% if %} tag evaluates a variable, and if that variable is “true” (i.e. exists, is not empty, and is not a false boolean value).

Syntax:-

```
{% if variable %}
        …………..
{% endif %}
```

Example:-

```
{% if nm %}
        </h1>Hello {{nm}}</h1>
{% endif %}


{% if nm and st %}
        </h1> For Course{{nm}} {{st}} Seat Available</h1>
{% endif %}
```

```
{% if nm or st %}
        </h1>Seat Available</h1>
{% endif %}

{% if not st%}
        </h1>Seat Not Available</h1>
{% endif %}
```

<u>if Tag with condition</u>

Syntax:-

{% if condition %}

…………..

{% endif %}

Example:-

{% if nm == ‘Django’ %}

</h1>Hello {{nm}}</h1>

{% endif %}

{% if nm == ‘Django’ and st==5 %}

</h1>{{nm}} Seat Available</h1>

{% endif %}

if tags may also use the operators **==, !=, <, >, <=, >=, in, not in, is,** and **is not**

{% if nm == ‘Django’ or st == 5 %}

</h1>{{nm}} Seat Available</h1>

{% endif %}

{% if not st == 5 %}

</h1>Seat Not Available</h1>

{% endif %}

if Tag with filter

Syntax:-

```
{% if variable|filter %}
        …………..
{% endif %}
```

Example:-

```
{% if nm|length >= 6 %}
        </h1>Hello {{nm}}</h1>
{% endif %}
```

if else Tag

Syntax:-

```
{% if variable %}
        …………..
{% else %}
        …………..
{% endif %}
```

Example:-

```
{% if nm %}
        </h1>Hello {{nm}}</h1>
{% else}
        <h1>No Course Available</h1>
{% endif %}
```

if else Tag with Condition

Syntax:-

```
{% if condition %}
        …………..
{% else %}
        …………..
{% endif %}
```

Example:-

```
{% if nm ==‘Django’ %}
        </h1>Hello {{nm}}</h1>
{% else}
        <h1>No Course Available</h1>
{% endif %}
```

if elif Tag

Syntax:-

```
{% if variable %}
        …………..
{% elif variable %}
        …………..
{% else %}
        …………..
{% endif %}
```

Example:-

```
{% if nm %}
        </h1>Hello {{nm}}</h1>
{% elif st %}
        </h1>Seats {{st}}</h1>
{% else}
        <h1>No Course Available</h1>
{% endif %}
```

if elif Tag with condition

Syntax:-

{% if condition %}

…………..

{% elif condition %}

…………..

{% else %}

…………..

{% endif %}

Example:-

{% if nm==‘Django’ %}

</h1>Hello {{nm}}</h1>

{% elif st==5 %}

</h1>Seats {{st}}</h1>

{% else}

<h1>No Course Available</h1>

{% endif %}

Dot Lookup

Technically, when the template system encounters a dot, it tries the following lookups, in this order:

- Dictionary lookup
- Attribute or method lookup
- Numeric index lookup

for loop Tag

Syntax:-

```
{% for variable in variables %}
    {{ variable }}
{% endfor %}
```

Example:-

```
<ul>
{% for stu in student %}
    <li>{{ stu }}</li>
{% endfor %}
</ul>
```

Syntax:-

```
{% for variable in variables %}
    {{ variable }}
{% empty %}
    Empty
{% endfor %}
```

Syntax:-

```
{% for key, value in data.items %}
    {{ key }}: {{ value }}
{% endfor %}
```

Predefined forloop Variable

Variable	Description
forloop.counter	The current iteration of the loop (1-indexed)
forloop.counter0	The current iteration of the loop (0-indexed)
forloop.revcounter	The number of iterations from the end of the loop (1-indexed)
forloop.revcounter0	The number of iterations from the end of the loop (0-indexed)
forloop.first	True if this is the first time through the loop
forloop.last	True if this is the last time through the loop
forloop.parentloop	For nested loops, this is the loop surrounding the current one

Example:-

```
{% for stu in student %}
   {{forloop.counter}}{{ stu }}
{% endfor %}
```